

Programación de microcontroladores

Timers

Juan José Londoño, M.Sc.

Introducción

En muchos sistemas embebidos se necesita que ciertas acciones ocurran después de cierto tiempo o de manera periódica.

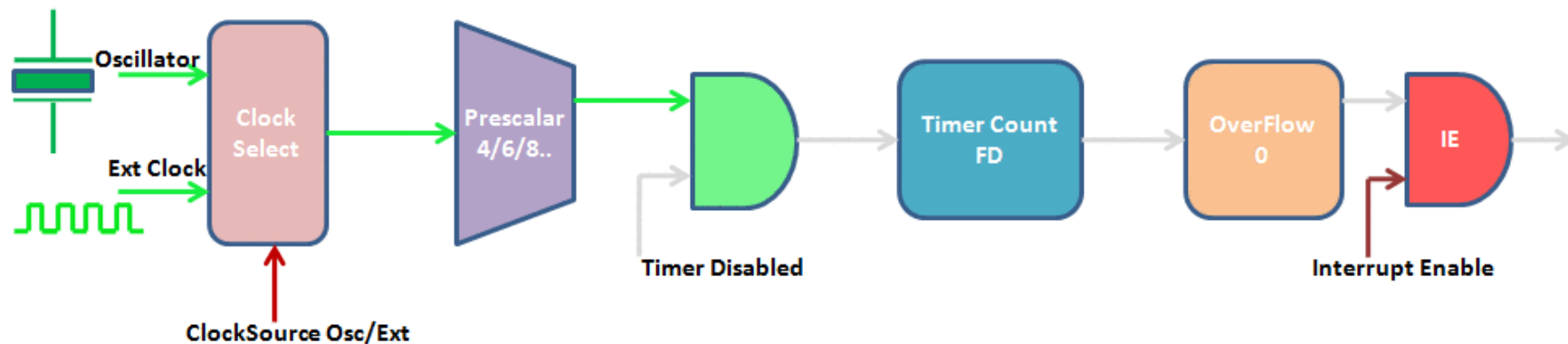
Para resolver esto, el microcontrolador necesita medir tiempo con precisión, todo ello sin bloquear por completo el funcionamiento de la CPU ni la ejecución del firmware.

Aquí aparecen los **timers**.

¿Qué es un Timer?

Un timer es un periférico hardware interno del microcontrolador que permite contar tiempo de forma automática.

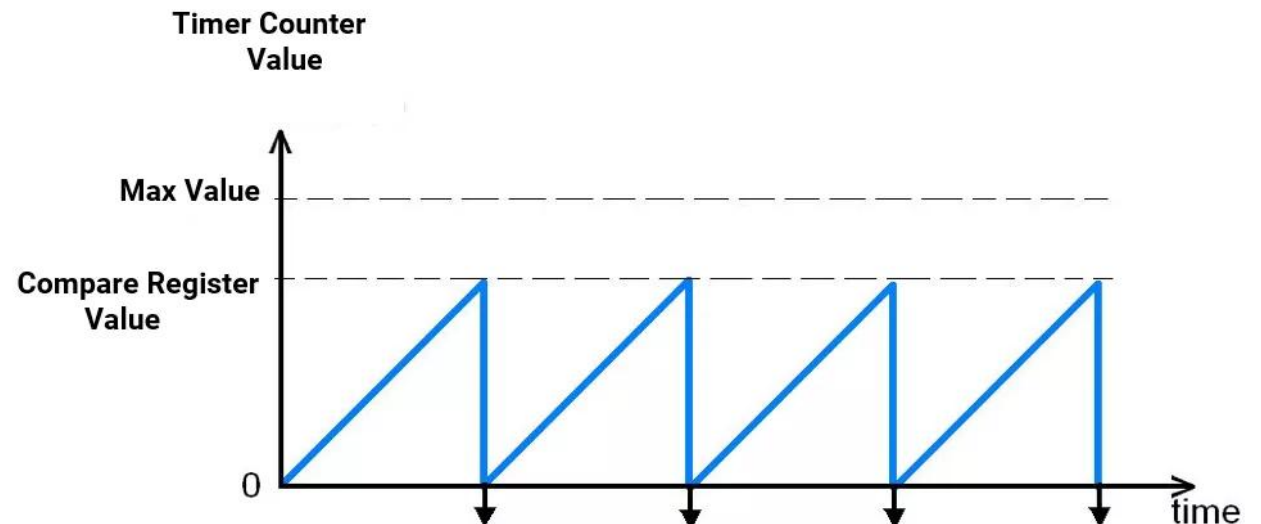
Internamente funciona como un contador que incrementa su valor con cada pulso de reloj.



¿Qué es un Timer?

Cuando el contador alcanza cierto valor pueden ocurrir cosas como:

- Reiniciarse.
- Activar una señal.
- Generar una interrupción.
- Compararse con otro valor.



Alternativa al *delay*

Los *delays* bloquean por completo a la CPU, lo único que hacen es “esperar”, mientras tanto el programa no hace nada más.

Mientras que un timer permite:

- Medir tiempo sin detener el programa.
- Ejecutar tareas periódicas.
- Generar eventos automáticos.

Elementos de un timer

Un timer tiene varios bloques importantes:

Clock Source:

El timer recibe una señal de reloj.

Puede venir de:

- Reloj del CPU.
- Reloj del sistema.
- Reloj dividido.

Elementos de un timer

Un timer tiene varios bloques importantes:

Prescaler:

El prescaler divide la frecuencia del reloj.

i EJEMPLO

Clock source = 80MHz

Prescaler = 80

Timer clock = 1MHz

1 tick = 1 μ s

Elementos de un timer

Un timer tiene varios bloques importantes:

Counter:

Es el registro que cuenta los ticks del reloj.

Compare/alarm:

El timer puede comparar el contador con un valor.

Entonces el timer puede reiniciarse, generar una interrupción o activar un evento.

Timers en el ESP32

El ESP32 tiene varios sistemas de temporización hardware. Dentro de ellos, 4 timers por hardware de propósito general, divididos en dos grupos, **Timer Group 0** y **Timer Group 1**, cada uno con timer 0 y 1.

Estos timers son periféricos independientes del CPU, por lo que pueden contar tiempo sin intervención del software.

Cada timer tiene un contador de 64 bits, por lo que pueden contar hasta:

$$2^{64} = 18.446.744.073.709.551.616$$

Timers en el ESP32

Modos de operación

Los timers pueden operar en dos modos:

- Ascendente.
- Descendente.

Auto-reload

Cuando el timer llega a cierto valor puede reiniciarse automáticamente o continuar contando, lo que permite crear **eventos periódicos**.

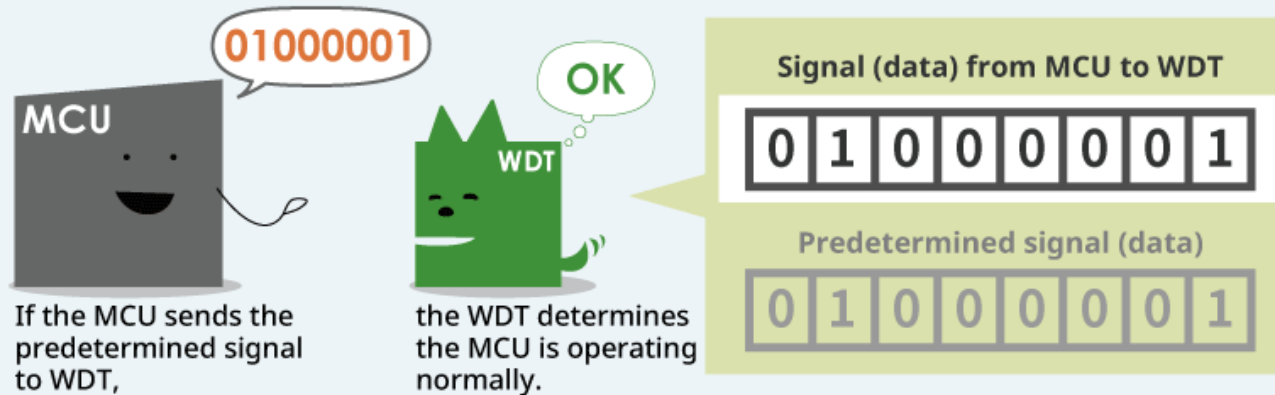
Watchdog timer (WDT)

Un watchdog timer es un timer especial que sirve para detectar cuando el sistema se bloquea.

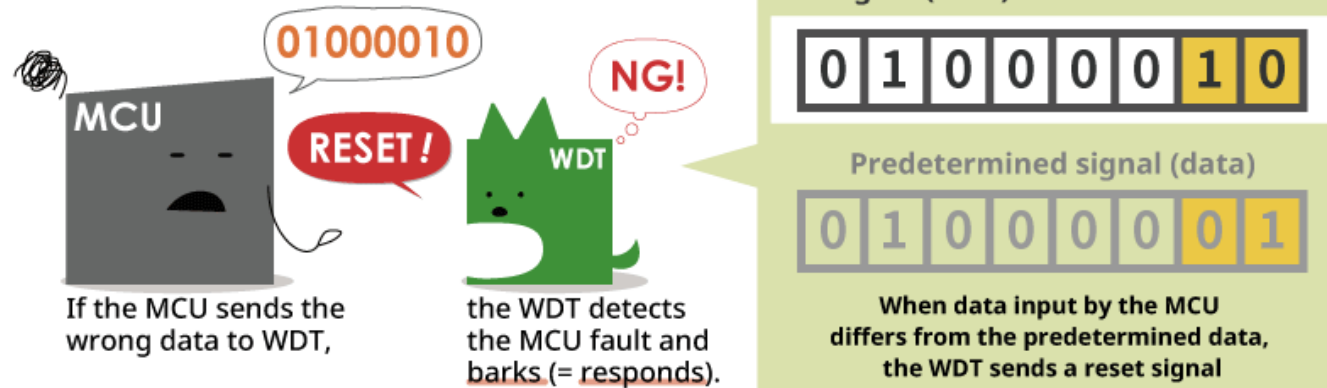
Funciona así:

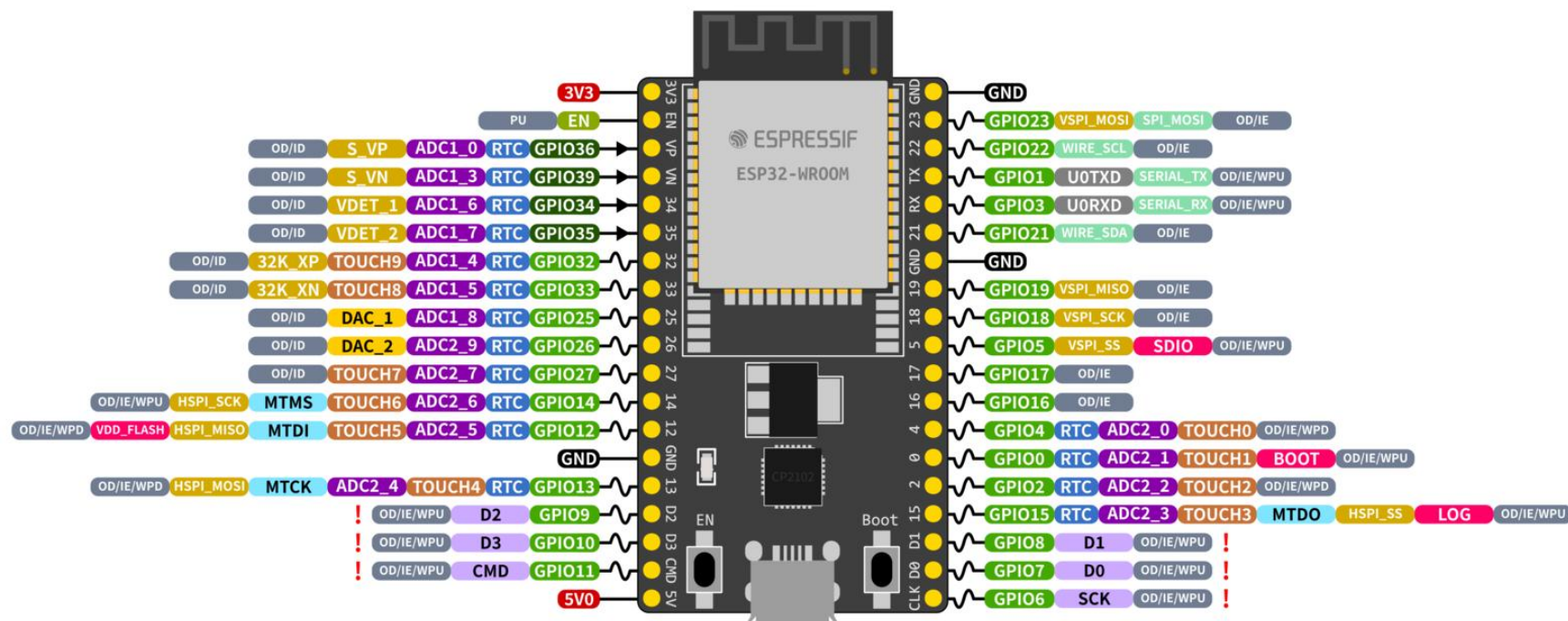
- Firmware debe "alimentar" el watchdog periódicamente.
- Si no lo hace, este asume que el sistema se bloqueó, por lo que reinicia el microcontrolador.

When the MCU is operating normally



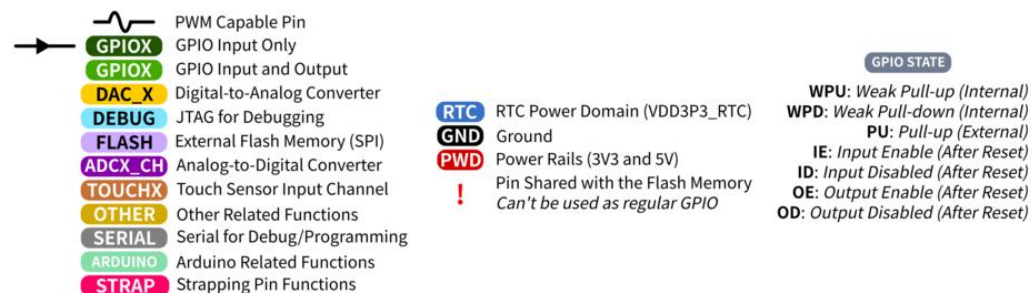
When the MCU is malfunctioning





ESP32 Specs

32-bit Xtensa® dual-core @240MHz
 Wi-Fi IEEE 802.11 b/g/n 2.4GHz
 Bluetooth 4.2 BR/EDR and BLE
 520 KB SRAM (16 KB for cache)
 448 KB ROM
 34 GPIOs, 4x SPI, 3x UART, 2x I2C,
 2x I2S, RMT, LED PWM, 1 host SD/eMMC/SDIO,
 1 slave SDIO/SPI, TWAI®, 12-bit ADC, Ethernet



Configuración de los timers



```
#include "freertos/FreeRTOS.h"  
#include "freertos/task.h"  
#include "driver/timer.h"
```



los timers del ESP32 están
conectados a un reloj de **80 MHz**

Configuración de los timers

```
timer_config_t config = {  
    .divider = 80, // 80 MHz/80 ⇒ 1 tick / 1 us  
    .counter_dir = TIMER_COUNT_UP,  
    .counter_en = TIMER_PAUSE,  
    .alarm_en = TIMER_ALARM_DIS,  
    .auto_reload = false,  
};  
  
timer_init(TIMER_GROUP_0, TIMER_0, &config);
```

Iniciar timer



```
timer_start(TIMER_GROUP_0, TIMER_0);
```


Leer el valor actual de un timer

```
uint64_t count = 0;  
timer_get_counter_value(TIMER_GROUP_0, TIMER_0, &count);
```

Reiniciar valor de un timer



```
timer_set_counter_value(TIMER_GROUP_0, TIMER_0, 0);
```

Detener un timer



```
timer_pause(TIMER_GROUP_0, TIMER_0);
```

Configuración del Watchdog Timer



```
#include "freertos/FreeRTOS.h"  
#include "freertos/task.h"  
#include "esp_task_wdt.h"
```

Configuración del Watchdog Timer

```
esp_task_wdt_config_t twdt_config = {  
    .timeout_ms = 3000,  
    .idle_core_mask = (1 << portNUM_PROCESSORS) - 1,  
    .trigger_panic = true,  
};  
  
esp_task_wdt_init(&twdt_config);  
esp_task_wdt_add(NULL);
```

Alimentar al WDT



```
// Alimenta el watchdog de esta tarea  
esp_task_wdt_reset();
```